

Ch4 字符串和格式化输入/输出

对应《C Primer Plus（第6版）》第4章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：位运算、指针高级用法

4.1 先把程序框架摆正

每一个 C 程序都长这样（先背下来，后面所有章节都在这个骨架里加东西）：

```
/* ch4_01_framework.c - 程序基本框架 */
#include <stdio.h>          /* 引入输入输出函数（printf/scanf 都在这里）*/

int main(void)              /* 程序入口：从这里开始执行 */
{
    printf("Hello, C!\n");   /* 输出一行文字 */
    return 0;               /* 告诉系统"正常结束" */
}
```

编译：gcc -Wall -Wextra -o hello.exe hello.c 运行：.\hello.exe

为什么要 -Wall -Wextra？ 它让编译器把「可疑但没报错」的写法也提示出来。初学者的 90% 低级错误（变量没初始化、类型不匹配）靠它提前暴露。养成习惯：写代码就开这两个选项，看到警告先修掉再运行。

4.2 printf：把东西"打印"到屏幕上

printf 的完整格式是 printf("格式字符串", 要输出的值们)。格式字符串里用 **%占位符** 表示「这里放一个值」，后面的值按顺序对号入座：

占位符	含义	示例输出
%d	整数 (int)	printf("%d", 42); → 42
%f	小数 (float/double, 默认 6 位小数)	printf("%f", 3.14); → 3.140000
%.2f	小数, 保留 2 位	printf("%.2f", 3.14159); → 3.14
%c	单个字符	printf("%c", 'A'); → A
%s	字符串	printf("%s", "hi"); → hi

还可以控制宽度：`%5d` 表示「至少占 5 个字符位，右对齐」；`%-5d` 左对齐。转义字符 `\n`（换行）、`\t`（制表）也在这里用。

```
/* ch4_02_printf.c - 格式化输出练习 */
#include <stdio.h>

int main(void)
{
    int    age  = 19;
    double temp = 36.5;

    printf("我 %d 岁\n", age);
    printf("体温 %.1f 度\n", temp);
    printf("%-8s|%-8s\n", "left", "right");    /* %-8s 左对齐占8位（ASCII 演示）*/
    return 0;
}
```

新手坑 4：中文对齐不按「字数」算。`%10s` 的宽度单位是**字节**，中文一个字 3 字节（UTF-8），"左对齐"3 个字已占 9 字节 → 想对齐得放大宽度，或用 ASCII 演示。上面示例特意用 "left"/"right" 演示对齐——换中文试试，竖线就对不齐了。

4.3 scanf：让程序"读"你输入的东西

`scanf` 和 `printf` 长得像，但有一个**最容易翻车**的差别：给变量传值时，变量名前面要加 **&**（取地址符号）。

```
/* ch4_03_scanf.c - 读取键盘输入 */
#include <stdio.h>

int main(void)
{
    int num;
    printf("请输入一个整数: ");
    scanf("%d", &num);    /* 注意 & ! 没有它程序会崩或乱读 */
    printf("你输入的是 %d\n", num);
    return 0;
}
```

新手坑 1：忘了 &。`scanf` 需要知道「把值存到哪个内存地址」，`%d` 配 `&num`。忘了写，轻则警告、重则段错误（程序崩溃）。

新手坑 2：%s 读不进空格。`scanf("%s", name)` 遇到空格就停，输入 "Li Ming" 只会读到 "Li"。要读一整行，用 `fgets(name, sizeof(name), stdin)`（第 11 章细讲）。

新手坑 3：缓冲区残留换行。连续 `scanf` 时，上一次按回车留下的 `\n` 可能被下一次 `scanf` 直接消费，表

现为「没让我输入就跳过了」。入门阶段先记住：`%c` 前面加个空格（`scanf(" %c", &ch)`）能吞掉残留换行。

4.4 字符串：其实就是字符数组

C 语言没有独立的「字符串类型」。字符串就是一个以 `\0`（空字符）结尾的 `char` 数组——最后一个元素一定是 `\0`，用来标记「字符串到这里结束」。

```
/* ch4_04_string.c - 字符串的本质 */
#include <stdio.h>

int main(void)
{
    char name[20] = "F-VIS"; /* 数组长度20，实际用了6个：F-V-I-S-\0 */
    printf("名称: %s\n", name);
    printf("第1个字符: %c\n", name[0]); /* 数组下标从 0 开始 */
    return 0;
}
```

一句话理解内存：数组名 `name` 本身是「数组起始位置的地址」。`name[0]` 是第一个元素、`name[1]` 是第二个……这段「地址 + 下标」的心智模型，是第 10 章指针的地基——现在先种下。

4.5 新手坑集中营（本章最容易错的 4 件事）

1. **`%d` 配 `double` / `%f` 配 `int`：**格式字符串和值类型对不上，输出是乱码。`printf` 里 `float` 会自动转 `double`，但 `scanf` 不会——`%f` 读 `float`，`%lf` 读 `double`。
2. **`scanf` 忘记 `&`**（见 4.3）。
3. **字符串数组不够大：**`char s[4] = "hello"`——GCC 直接报警告 `initializer-string too long`，而且数组里没有 `\0` 的位置，`%s` 打印会越界读内存。开 `-Wall -Wextra` 让编译器提前拦住它。
4. **`\n` 忘写：**输出全部挤在一行，看不出结构。

4.6 一分钟自测（答案见练习册）

判断对错：

- ① `printf("%d", 3.14);` 是合法的，会正常输出 3。
- ② `scanf("%d", num);` 是正确的读入写法。
- ③ 字符串 "OK" 实际占 3 个 `char` 的位置（含 `\0`）。

4.7 练习指引

做完本章内容后，去练习册完成 Ch4 配套练习；同时打开《C Primer Plus（第6版）》第 4 章，完成**复习题 1、3、5**（题号对应主教材）。改错题是练习册里最有价值的部分——先自己找 bug，再看答案的思路讲解。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。